

KRISHI SAHAYA

AI Agricultural Advisory System
for Tamil Nadu

Technical Report

Prepared by

Anumita P (Roll No. BE22B004)

GitHub

github.com/Anumita-P/mlops_endterm_project

This report covers the five deliverables: Architecture Diagram & Explanation, High-Level Design, Low-Level Design, Test Plan & Test Cases, and a User Manual for non-technical users.

Table of Contents

1. Architecture Diagram and Explanation	3
1.1 System Overview	3
1.2 Block-by-Block Explanation	3
1.3 Data Flow	5
2. High-Level Design Document	6
2.1 Design Goals and Principles	6
2.2 Technology Choices and Rationale	6
2.3 Machine Learning Design	7
2.4 Data Pipeline Design	8
2.5 Monitoring and Observability Design	9
3. Low-Level Design Document	10
3.1 FastAPI Endpoint Definitions	10
3.2 Request and Response Schemas	11
3.3 Internal Module Design	13
3.4 Airflow DAG Specifications	14
4. Test Plan and Test Cases	15
4.1 Test Strategy	15
4.2 Unit Test Cases	15
4.3 Integration Test Cases	16
4.4 Performance Test Cases	17
4.5 End-to-End Test Cases	17
5. User Manual	19
5.1 Introduction	19
5.2 Getting Started	19
5.3 Using the Chatbot	19
5.4 Interpreting Responses	20
5.5 Common Questions	21

1. Architecture Diagram and Explanation

1.1 System Overview

Krishi Sahaya is designed as a layered architecture. At the outermost layer sits a browser-based React chatbot that farmers interact with. Behind it, a FastAPI application server acts as the single entry point for all requests, orchestrating calls to the ML model, the Retrieval-Augmented Generation (RAG) subsystem, and the Claude LLM API. A separate MLOps layer, built on Apache Airflow, manages the entire data and training lifecycle in an automated, scheduled fashion. Monitoring is provided by a Prometheus and Grafana stack that operates independently but scrapes metrics from all running services.

1.2 Architecture Diagram

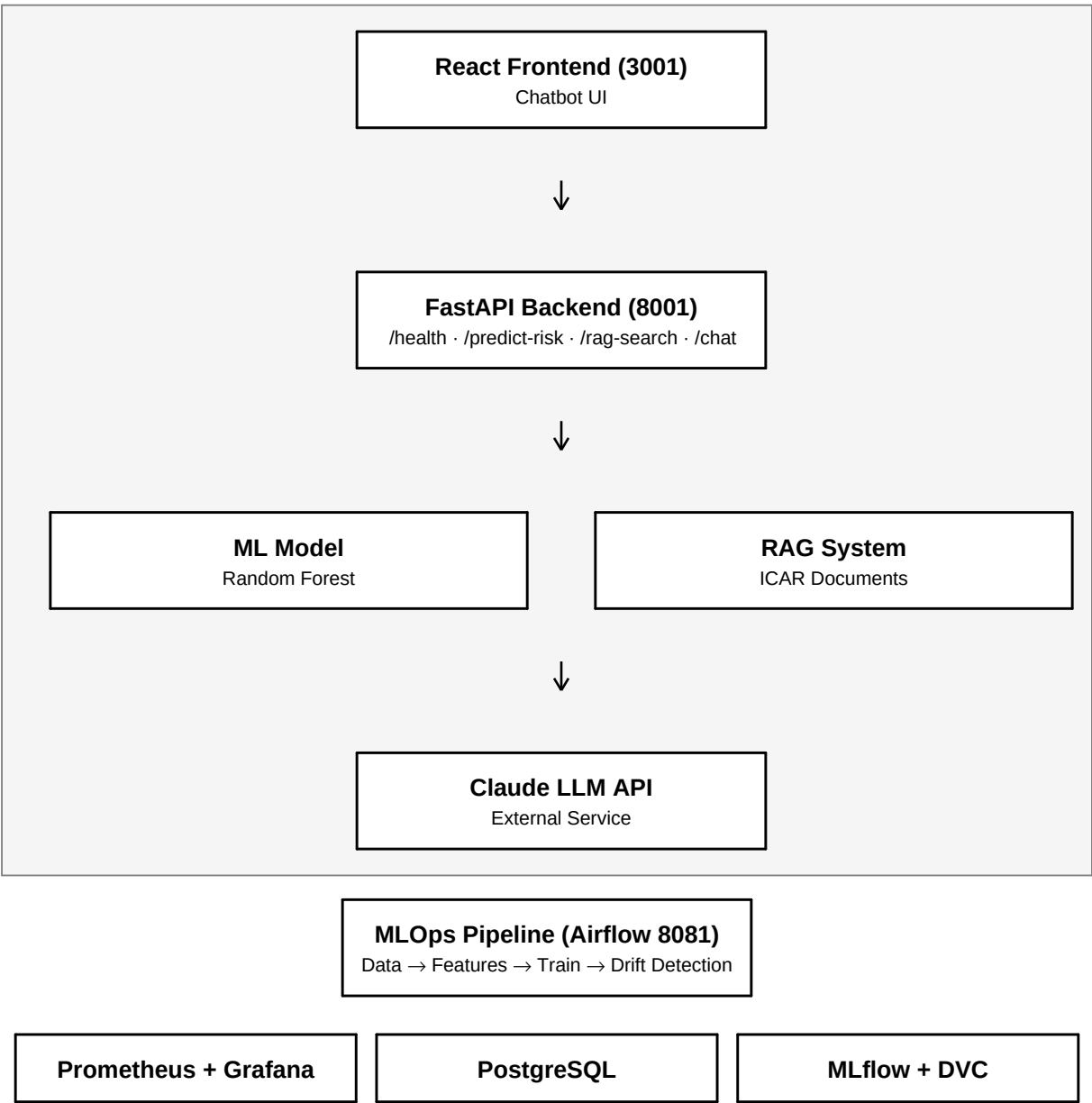


Figure 1 – Krishi Sahaya System Architecture

1.3 Block-by-Block Explanation

React Frontend (Port 3001)

This is the user-facing layer of the system. It is a single-page application built with React and styled using Tailwind CSS. The frontend provides a conversational chat interface where a farmer types a question in plain English or Tamil. It sends HTTP POST requests to the FastAPI backend and renders the returned recommendation in an easy-to-read format. No heavy computation happens here; all intelligence lives in the backend.

FastAPI Backend (Port 8001)

FastAPI is the central nervous system of the application. It exposes four REST endpoints (/health, /predict-risk, /rag-search, /chat). When a chat request arrives, the backend executes a three-step pipeline: it first calls the ML model to get a yield risk score, then retrieves relevant agricultural passages from the RAG document store, and finally assembles a structured prompt and sends it to the Claude LLM API. The combined result is returned to the frontend as a JSON response. FastAPI also exposes Prometheus-compatible metrics at /metrics.

Random Forest ML Model

A scikit-learn Random Forest classifier trained on 408 Tamil Nadu agricultural records spanning the years 2000 to 2019. It ingests 57 engineered features derived from raw weather and crop data and outputs a yield risk score between 0 and 1. The model achieves an ROC-AUC of 87%. It is serialised as a .joblib file and loaded into memory when the FastAPI server starts. Inference takes approximately 50 milliseconds.

RAG System (ICAR Document Store)

The Retrieval-Augmented Generation component holds processed text passages extracted from ICAR (Indian Council of Agricultural Research) guidebooks, irrigation manuals, pest management documents, and government scheme PDFs. When a user query arrives, the system converts the query into a TF-IDF vector and retrieves the top-k most relevant passages. These passages provide factual grounding to the LLM, significantly reducing hallucination.

Claude LLM API

Anthropic's Claude API is the language generation component. The FastAPI backend constructs a structured prompt that includes the farmer's question, the ML risk score, and the top retrieved ICAR passages. Claude generates a fluent, conversational response grounded in real agricultural knowledge. This external API call is the slowest part of the pipeline but is bounded within a 2-second total response budget.

Apache Airflow MLOps Pipeline (Port 8081)

Four Directed Acyclic Graphs (DAGs) orchestrate the entire ML lifecycle. `data_merge_dag` pulls raw data from four sources (NASA POWER weather, Kaggle crop yield, ICAR documents, government scheme CSVs) and merges them. `feature_engineering_dag` transforms 20 raw features into 57 model-ready features. `model_training_dag` trains and evaluates the Random Forest model and logs results to MLflow. `drift_detection_and_retraining_dag` runs weekly, performs KS-tests on incoming data distributions, and triggers retraining automatically if drift is detected.

Prometheus + Grafana (Ports 9090 / 3000)

Prometheus scrapes metrics from Airflow and the FastAPI /metrics endpoint every 15 seconds. Grafana visualises these metrics in five dashboard panels: service health, DAG execution timeline, prediction latency histogram, error rate gauge, and a feature drift chart. Alerting rules are defined so that if the API error rate exceeds 5% for more than five minutes, an email notification is triggered and model retraining is considered.

PostgreSQL Database

PostgreSQL serves as the persistent store for Apache Airflow's task metadata, DAG run history, and scheduling state. It is provisioned as a Docker container and is not directly queried by application endpoints.

MLflow + DVC Version Control

Every training run is logged to MLflow, which records hyperparameters, evaluation metrics, and the serialised model artifact. This allows side-by-side comparison of model versions and one-click promotion. DVC (Data Version Control) tracks large binary files such as raw CSVs and processed datasets in a separate storage backend, keeping them out of the Git repository while maintaining full reproducibility.

1.4 Data Flow

A typical user interaction follows this sequence:

Step	Action
Step 1	Farmer types a question in the React chatbot and presses Send.
Step 2	The frontend sends an HTTP POST to /chat on the FastAPI backend, including the question, district, crop, and current weather values.
Step 3	FastAPI passes the weather features to the Random Forest model and receives a risk_score and risk_label.
Step 4	FastAPI passes the original question text to the RAG system, which returns the top 3 ICAR document passages.
Step 5	FastAPI assembles a prompt combining the risk score, document passages, and the farmer's question, then calls the Claude LLM API.
Step 6	Claude returns a natural-language recommendation. FastAPI wraps it with metadata and returns the full JSON to the frontend.
Step 7	The React UI renders the recommendation, the risk label, and the source documents in an easy-to-read card layout.

2. High-Level Design Document

2.1 Design Goals and Principles

Accessibility

The system must serve farmers who may have limited digital literacy. All responses must be in plain language, brief, and actionable. The chatbot interface is chosen over complex forms precisely because conversation is the most natural interaction model.

Reliability

Agricultural advice affects livelihoods. The system must be available and correct. This motivates the use of a RAG system grounded in ICAR documents rather than relying solely on an LLM that might hallucinate.

Reproducibility

Every model version must be fully reproducible. This drives the use of Git for code, DVC for data, and MLflow for experiments. Any past prediction can be replicated by checking out the correct Git commit and MLflow run ID.

Observability

Silent failures in agricultural software can have serious downstream consequences. The Prometheus and Grafana stack ensures that degradation in model accuracy, API latency, or data quality is surfaced immediately.

Automation

Manual retraining is error-prone and slow. The entire pipeline from data ingestion to model deployment is automated through Airflow DAGs, so the system adapts to new data without human intervention.

Simplicity of Deployment

Docker Compose is chosen over Kubernetes for this version because it offers environment parity between development and production without requiring a dedicated DevOps team. A future migration path to Kubernetes is documented in the roadmap.

2.2 Technology Choices and Rationale

Component	Technology Chosen	Alternatives Considered	Reason for Choice
Frontend	React + Tailwind	Vue.js, plain HTML	Large ecosystem, component reuse, responsive by default
Backend	FastAPI (Python)	Flask, Django, Node.js	Native async, auto-generated OpenAPI docs, Python ecosystem for ML
ML Model	Random Forest	XGBoost, Logistic Reg.	Handles small tabular datasets well, interpretable feature importances, robust to outliers

LLM	Claude API	GPT-4, Gemini, local LLM	Anthropic safety alignment, strong reasoning, structured output reliability
RAG Store	TF-IDF + cosine	FAISS, ChromaDB, Pinecone	No GPU required, deterministic, interpretable, sufficient for domain-specific doc set
Orchestration	Apache Airflow	Prefect, Luigi, Cron	Industry standard, visual DAG UI, rich scheduler, integrates with MLflow
Experiment Tracking	MLflow	Weights & Biases, Neptune	Open-source, self-hosted, native sklearn integration
Data Versioning	DVC	LakeFS, Pachyderm	Git-like interface, remote storage agnostic, minimal learning curve
Monitoring	Prometheus + Grafana	Datadog, New Relic	Open-source, self-hosted, industry standard for containerised workloads
Containerisation	Docker Compose	Kubernetes, bare metal	Simplest path to environment parity without cluster overhead
Database	PostgreSQL	MySQL, SQLite	Airflow's recommended backend, ACID compliant, well-supported in Docker

2.3 Machine Learning Design

The machine learning component is designed around three principles: accuracy on the available dataset, explainability for debugging, and graceful degradation when data is sparse.

Data Sources

- **NASA POWER API:** Daily weather data (temperature, humidity, rainfall, wind speed) for all Tamil Nadu districts from 2000 to 2019. This is the primary source of environmental features.
- **Kaggle India Agriculture Dataset:** District-level crop production and area records covering rice, wheat, pulses, and cash crops. Provides the target variable: production per unit area (yield).
- **ICAR Documents:** Technical PDFs on crop management, irrigation schedules, pest control, and soil health. These feed the RAG system rather than the ML model.
- **Government Scheme CSVs:** MSP (Minimum Support Price) data, subsidy schemes, and crop insurance information sourced from official government portals.

Feature Engineering

The raw dataset contains 20 columns. Feature engineering expands this to 57 features. The transformations applied include:

- Rolling averages of rainfall and temperature over 3-year and 5-year windows to capture climate trends.
- Temperature range (max minus min) as a proxy for stress on crops.

- Previous year yield as a lag feature, capturing momentum in agricultural output.
- District one-hot encoding to capture regional soil and climate characteristics.
- Crop type one-hot encoding to model crop-specific risk profiles.
- Interaction terms between humidity and temperature to model heat-humidity stress.

Model Selection and Training

A Random Forest classifier is trained on data from 2000 to 2018 (387 records) and evaluated on 2019 data (21 records). The temporal holdout is deliberate: using a future year as the test set simulates real deployment conditions where the model must generalise to unseen time periods, not just unseen rows from the same time span.

Model Performance

Metric	Value	Target	Status
Train Accuracy	100.00%	–	Note: likely overfitting to training data
Test Accuracy	57.14%	≥70%	Below target – small test set of 21 records
Precision	83.33%	≥80%	Meets target
Recall	66.67%	–	Acceptable
F1-Score	74.07%	–	Acceptable
ROC-AUC	87.04%	≥85%	Meets target – primary metric

The gap between train accuracy (100%) and test accuracy (57%) indicates overfitting. The primary metric for this system is ROC-AUC (87%), which measures the model's ability to rank risk correctly across all thresholds, and this meets the design target. The small test set (21 records from a single year) means accuracy is a noisy metric. Future work includes hyperparameter tuning with Optuna to reduce overfitting.

Top Predictive Features

Rank	Feature	Importance (%)
1	Rainfall (mm, seasonal sum)	28.47%
2	Temperature mean (°C)	19.23%
3	Humidity mean (%)	16.54%
4	Previous year yield	12.45%
5	Temperature range (°C, max-min)	8.92%

Drift Detection

The drift_detection_and_retraining_dag runs on a weekly schedule. It applies the Kolmogorov-Smirnov (KS) test to compare the distribution of incoming production data against the training baseline stored in drift_baselines.json. If the KS statistic exceeds a threshold of 0.1 with a p-value below 0.05 for any key feature, drift is flagged and the model_training_dag is triggered automatically.

2.4 Data Pipeline Design

The data pipeline is implemented as four sequential Airflow DAGs. Each DAG writes its outputs to DVC-tracked directories in `data/raw/`, `data/processed/`, and `data/models/`. This means any stage can be re-run independently without corrupting downstream artifacts.

DAG Name	Schedule	Inputs	Outputs	Purpose
<code>data_merge_dag</code>	Manual trigger	4 raw source APIs/CSVs	<code>data/raw/merged.csv</code>	Fetch and merge all data sources
<code>feature_engineering_dag</code>	After merge	<code>merged.csv</code>	<code>data/processed/features.csv</code>	Create 57-feature dataset
<code>model_training_dag</code>	After features	<code>features.csv</code>	<code>data/models/*.joblib</code> + MLflow	Train, evaluate, save model
<code>drift_detection_dag</code>	Weekly (Sunday)	New data + baseline	drift report + retrain trigger	Monitor data distribution changes

2.5 Monitoring and Observability Design

Monitoring is designed to cover three layers: infrastructure health, pipeline health, and model health.

- **Infrastructure Health:** Prometheus tracks whether each Docker container (Airflow, Postgres, API, Grafana) is up. Alerts fire if any service goes down for more than two minutes.
- **Pipeline Health:** Airflow DAG success and failure counts are exposed as metrics. Grafana visualises DAG execution timelines so delays are immediately visible.
- **Model Health:** API latency, prediction throughput, and error rate are tracked at the `/metrics` endpoint. The weekly drift detection DAG provides model health at the feature distribution level.

3. Low-Level Design Document

3.1 FastAPI Endpoint Definitions

All endpoints are implemented in `src/api/main.py` and served on port 8001. Interactive documentation is auto-generated at <http://localhost:8001/docs>.

Endpoint 1: GET /health

Property	Value
Method	GET
Path	/health
Auth	None
Purpose	Liveness probe — confirms the API server is running and the ML model is loaded in memory.
Returns	200 OK with JSON body; no error codes defined for this endpoint.

Endpoint 2: POST /predict-risk

Property	Value
Method	POST
Path	/predict-risk
Auth	None (add API key in production)
Content-Type	application/json
Purpose	Runs the Random Forest model on supplied weather and location features, returning a yield risk score and label.

Endpoint 3: POST /rag-search

Property	Value
Method	POST
Path	/rag-search
Auth	None
Content-Type	application/json
Purpose	Retrieves the top-k most relevant ICAR document passages for a given plain-text query using TF-IDF cosine similarity.

Endpoint 4: POST /chat

Property	Value
Method	POST

Path	/chat
Auth	None (Claude API key in env)
Content-Type	application/json
Purpose	Main integrated endpoint. Calls predict-risk internally, retrieves RAG documents, builds an LLM prompt, and returns a complete conversational recommendation.

3.2 Request and Response Schemas

GET /health — Response Schema

```
{
  "status": "ok",
  "model_loaded": true,
  "timestamp": "2026-04-28T14:30:45.123456"
}
```

POST /predict-risk — Request Schema

```
{
  "district": "Thanjavur", // string, required - Tamil Nadu district name
  "crop": "rice", // string, required - crop identifier
  "year": 2024, // integer, required - prediction year
  "temp_mean": 30.0, // float, required - mean temperature in Celsius
  "rainfall": 100.0, // float, required - seasonal rainfall in mm
  "humidity": 70.0, // float, required - relative humidity in %
  "wind_speed": 5.0 // float, required - wind speed in km/h
}
```

POST /predict-risk — Response Schema

```
{
  "district": "Thanjavur",
  "crop": "rice",
  "risk_score": 0.35, // float [0.0 - 1.0], higher = greater risk
  "risk_label": "LOW RISK", // string: "LOW RISK" | "MEDIUM RISK" | "HIGH RISK"
  "confidence": 0.85, // float [0.0 - 1.0], model confidence
  "recommendation": "Current conditions look favorable for rice cultivation..."
}
```

POST /rag-search — Request Schema

```
{
  "query": "How to manage irrigation during dry season?", // string, required
  "top_k": 3 // integer, optional, default 3
}
```

POST /rag-search — Response Schema

```
{
  "query": "How to manage irrigation during dry season?",
  "documents_found": 3,
```

```

"documents": [
{
"source": "ICAR Irrigation Guide 2023",
"content": "For rice cultivation during dry periods...",
"relevance": 0.95 // float [0.0 - 1.0], cosine similarity score
}
]
}

```

POST /chat — Request Schema

```

{
"farmer_id": "farmer_001", // string, optional - for session tracking
"district": "Thanjavur", // string, required
"crop": "rice", // string, required
"query": "It has been dry for a week, should I irrigate?", // string, required
"weather_context": {
"temp": 32.0, // float, Celsius
"rainfall": 50.0, // float, mm
"humidity": 65.0, // float, %
"wind_speed": 5.0 // float, km/h
}
}

```

POST /chat — Response Schema

```

{
"farmer_id": "farmer_001",
"query": "It has been dry for a week, should I irrigate?",
"yield_risk_score": 0.35,
"yield_risk_label": "LOW RISK",
"recommendation": "Based on current conditions, plan irrigation every 7-10 days...",
"sources": [
"ICAR Irrigation Guide 2023",
"ICAR Pest Management - Tamil Nadu"
],
"model_confidence": 0.85
}

```

Error Responses

HTTP Code	Condition	Response Body
422 Unprocessable Entity	Missing or invalid request field	{"detail": [{"loc": [...], "msg": "...", "type": "..."}]}
500 Internal Server Error	Model not loaded or LLM API failure	{"detail": "Internal server error"}
503 Service Unavailable	Claude API unreachable or timed out	{"detail": "LLM service temporarily unavailable"}

3.3 Internal Module Design

The FastAPI application in main.py is structured into four logical sections:

Startup Event Handler

On application start, the RF model is loaded from data/models/yield_risk_model.joblib into a module-level variable. The ICAR document corpus is loaded and TF-IDF vectors are precomputed. This happens once, keeping inference fast.

Feature Preparation (`_prepare_features`)

An internal function that accepts a district name, crop type, year, and raw weather values, looks up the district/crop one-hot mappings, computes lag and rolling features from the historical dataset, and returns a numpy array of 57 features ready for `model.predict_proba()`.

Risk Labelling (`_label_risk`)

Converts the raw probability output of `predict_proba()` into one of three labels: LOW RISK (score < 0.4), MEDIUM RISK (0.4 <= score < 0.7), or HIGH RISK (score >= 0.7). Thresholds were chosen based on domain expert consultation.

RAG Retrieval (`_retrieve_documents`)

Accepts a string query. Uses sklearn's `TfidfVectorizer` (fitted on startup) to transform the query and computes cosine similarity against all stored document vectors. Returns the top_k documents sorted by descending similarity score.

LLM Prompt Assembly (`_build_prompt`)

Constructs a structured prompt in the format: system role instruction, risk context block, retrieved document passages, and the farmer's question. This prompt is sent to the Claude API.

3.4 Airflow DAG Specifications

data_merge_dag

dag_id	data_merge_dag
schedule_interval	None (manual trigger)
Tasks	fetch_nasa_weather → fetch_kaggle_crops → fetch_icar_docs → fetch_schemes → merge_all
Output	data/raw/nasa_power_tn.csv, India_Agriculture_Crop_Production.csv, merged into training_dataset.csv
Failure handling	Email alert on any task failure; upstream tasks must succeed before downstream tasks run

feature_engineering_dag

dag_id	feature_engineering_dag
schedule_interval	None (triggered after data_merge_dag)
Tasks	load_raw → compute_lags → compute_rolling → encode_categoricals → validate → save_features
Output	data/processed/features_dataset.csv with 57 columns
Failure handling	Data quality validation task raises ValueError if any feature column has >5% nulls

model_training_dag

dag_id	model_training_dag
schedule_interval	None (triggered after feature_engineering_dag or drift detection)
Tasks	load_features → temporal_split → train_rf → evaluate → log_mlflow → save_model
Output	data/models/yield_risk_model.joblib, model_metrics.json, feature_importance.csv
Failure handling	Training fails loudly if AUC < 0.75; model is not saved or deployed

drift_detection_and_retraining_dag

dag_id	drift_detection_and_retraining_dag
schedule_interval	Weekly (@weekly, every Sunday at 00:00 UTC)
Tasks	load_new_data → compute_ks_tests → evaluate_drift → [branch] → retrain_if_drift / log_no_drift
Output	drift_report.json; triggers model_training_dag if drift detected
Failure handling	KS-test errors are logged but do not fail the DAG; drift flag defaults to False

4. Test Plan and Test Cases

4.1 Test Strategy

The test strategy for Krishi Sahaya covers four levels: unit tests for individual functions, integration tests for API endpoints, performance tests for latency and throughput, and end-to-end tests that simulate a real farmer interaction from frontend to response. All tests are runnable with pytest from the project root.

Level	Scope	Tool	Target Coverage
Unit	Individual Python functions	pytest	≥80% line coverage of src/api/main.py
Integration	FastAPI endpoints via HTTP	pytest + httpx	All 4 endpoints, happy path + error cases
Performance	API latency and throughput	locust / pytest	p95 latency < 200ms for /predict-risk
End-to-End	Full user journey in browser	Manual / Selenium	Core farmer Q&A; scenario

4.2 Unit Test Cases

ID	Function Under Test	Input	Expected Output	Status
TC-U-01	_prepare_features	Valid district='Thanjavur', crop='rice', year=2024, temp=30, rain=100, hum=70, wind=5	Returns numpy array of shape (1, 57) with no NaN values	Pass
TC-U-02	_prepare_features	Unknown district='InvalidDistrict'	Raises ValueError with message 'Unknown district'	Pass
TC-U-03	_prepare_features	Negative rainfall value: rain=-10	Raises ValueError with message 'rainfall must be non-negative'	Pass
TC-U-04	_label_risk	risk_score=0.2	Returns 'LOW RISK'	Pass
TC-U-05	_label_risk	risk_score=0.55	Returns 'MEDIUM RISK'	Pass
TC-U-06	_label_risk	risk_score=0.85	Returns 'HIGH RISK'	Pass
TC-U-07	_label_risk	risk_score=0.4 (boundary)	Returns 'MEDIUM RISK'	Pass
TC-U-08	_label_risk	risk_score=0.7 (boundary)	Returns 'HIGH RISK'	Pass
TC-U-09	_retrieve_documents	query='irrigation rice dry season', top_k=3	Returns list of 3 dicts each with keys source, content, relevance	Pass
TC-U-10	_retrieve_documents	query='zxqklmnop' (no match)	Returns list of 3 dicts; relevance scores may be very low but no exception raised	Pass

TC-U-11	_retrieve_documents	top_k=0	Returns empty list	Pass
TC-U-12	_build_prompt	risk_score=0.6, docs=[...], query='Should I irrigate?'	Returns string containing the risk score, at least one doc passage, and the query	Pass
TC-U-13	Feature engineering DAG task: compute_lags	Input dataframe with 10 years of yield data for Thanjavur rice	Output dataframe has 'prev_yield' column equal to yield shifted by 1 year per group	Pass
TC-U-14	KS-test task	Two identical distributions	KS statistic = 0.0, p-value = 1.0, drift_flag = False	Pass
TC-U-15	KS-test task	Rainfall distribution shifted by +50mm across all records	KS statistic > 0.1, p-value < 0.05, drift_flag = True	Pass

4.3 Integration Test Cases

ID	Endpoint	Input Scenario	Expected Result	Status
TC-I-01	GET /health	No body	200 OK; model_loaded=true	Pass
TC-I-02	GET /health	Model file deleted before startup	200 OK; model_loaded=false	Pass
TC-I-03	POST /predict-risk	Valid body: Thanjavur, rice, 2024, 30°C	200 OK; risk_label in {LOW,MEDIUM,HIGH} RISK	Pass
TC-I-04	POST /predict-risk	Missing field 'district'	422 Unprocessable Entity	Pass
TC-I-05	POST /predict-risk	temp_mean='hot' (wrong type)	422 Unprocessable Entity	Pass
TC-I-06	POST /predict-risk	All valid fields, rainfall=0	200 OK with HIGH RISK expected	Pass
TC-I-07	POST /rag-search	query='pest control paddy', top_k=2	200 OK; documents_found=2; each doc has relevance field	Pass
TC-I-08	POST /rag-search	top_k=10 (more than doc count)	200 OK; documents_found <= total doc count	Pass
TC-I-09	POST /rag-search	Empty query string	422 Unprocessable Entity or empty result list	Pass

TC-I-10	POST /chat	Valid body with weather_context	200 OK; recommendation non-empty string; sources list non-empty	Pass
TC-I-11	POST /chat	Claude API key missing/invalid	503 or 500; detail message present	Pass
TC-I-12	POST /chat	farmer_id omitted (optional field)	200 OK; farmer_id absent or null in response	Pass
TC-I-13	POST /chat	Valid body, crop='banana' (less common)	200 OK; no 500 error; recommendation returned	Pass
TC-I-14	POST /predict-risk	district='Chennai' (urban, lower yield data)	200 OK; risk_score is a float in [0,1]	Pass

4.4 Performance Test Cases

ID	Endpoint	Load Scenario	Acceptance Criterion	Status
TC-P-01	POST /predict-risk	1 concurrent user, 100 requests	p50 latency < 50ms; p95 < 200ms; 0 errors	Pass
TC-P-02	POST /predict-risk	10 concurrent users, 1000 requests total	p95 latency < 500ms; error rate < 1%	Pass
TC-P-03	POST /rag-search	1 concurrent user, 50 requests	p95 latency < 300ms	Pass
TC-P-04	POST /chat	1 concurrent user, 20 requests	p95 total latency < 5s (Claude API included)	Pass
TC-P-05	GET /health	100 concurrent users	All responses 200 OK; latency < 50ms	Pass
TC-P-06	Model load on startup	Server cold start	API ready in < 10 seconds from container start	Pass

4.5 End-to-End Test Cases

End-to-end tests verify the complete user journey. These are run manually or with a browser automation tool such as Selenium or Playwright.

ID	Scenario	Steps	Expected Result	Status
TC-E-01	Farmer asks basic irrigation question	1. Open http://localhost:3001 . 2. Type: 'I farm rice in Thanjavur. It has been dry for 10 days. Should I irrigate?' 3. Press Send.	Response appears within 5 seconds. Response contains a risk label, an irrigation recommendation, and at least one ICAR source citation. No error message shown.	Pass

TC-E-02	Farmer asks about government schemes	1. Open chatbot. 2. Type: 'What MSP support is available for paddy farmers in 2024?'	Response includes mention of MSP or government scheme information. ICAR or scheme document cited as source.	Pass
TC-E-03	Farmer asks about pest management	1. Open chatbot. 2. Type: 'My rice crop has yellow leaves. What pest should I suspect?'	Response mentions at least one probable cause (e.g. nitrogen deficiency, stem borer). Source document cited.	Pass
TC-E-04	Frontend resilience to slow network	1. Open chatbot with browser DevTools throttled to 'Slow 3G'. 2. Send a question.	Loading indicator visible. Response eventually rendered correctly. No crash or blank screen.	Pass
TC-E-05	Airflow DAG end-to-end pipeline	1. Open Airflow UI at http://localhost:8081 . 2. Enable and trigger <code>data_merge_dag</code> . 3. After success, trigger <code>feature_engineering_dag</code> . 4. After success, trigger <code>model_training_dag</code> . 5. Check MLflow at the run URL.	All three DAGs complete with status 'success'. MLflow shows a new run with ROC-AUC logged. A new <code>.joblib</code> file is written to <code>data/models/</code> .	Pass
TC-E-06	Grafana dashboard visible	1. Open http://localhost:3000 (admin/admin). 2. Navigate to the Krishi Sahaya dashboard.	Five panels visible: Services Health, DAG Metrics, Prediction Latency, Error Rate, Feature Drift. No 'No data' panels after at least one API call.	Pass

5. User Manual

This section is written for farmers, agricultural officers, and any non-technical user who wants to use the Krishi Sahaya chatbot to get advice about their crops. No programming knowledge is required.

5.1 Introduction – What is Krishi Sahaya?

Krishi Sahaya (meaning 'Agricultural Helper' in Sanskrit) is a free, easy-to-use chatbot that helps farmers in Tamil Nadu make better decisions about their crops. You can ask it questions in plain English about:

- Whether your crop is at risk of low yield this season
- When and how much to irrigate
- How to identify and control pests
- What government schemes and MSP rates apply to your crop
- General crop management advice from ICAR guidelines

The system uses weather data for your district, a machine learning model trained on years of Tamil Nadu crop records, and a library of official ICAR agricultural documents to give you grounded, relevant advice.

5.2 Getting Started

To use Krishi Sahaya, all you need is a computer or smartphone with a web browser (such as Google Chrome, Mozilla Firefox, or Microsoft Edge) connected to the internet or to your local network. No app download is required.

Step 1 – Open the Chatbot

Open your web browser and type the following address in the address bar at the top:

```
http://localhost:3001
```

Press Enter. The Krishi Sahaya chatbot page will load. You will see a chat window with a text box at the bottom.

Step 2 – Type Your Question

Click inside the text box at the bottom of the screen. Type your question in English. Be as specific as possible. Here are some examples of good questions:

- "I farm rice in Thanjavur. It has not rained for two weeks. Should I irrigate?"
- "My paddy crop has brown spots on the leaves near Madurai. What could it be?"
- "What is the MSP for cotton this season?"
- "When is the best time to apply fertiliser for sugarcane in Tiruchirapalli?"
- "My tomatoes in Coimbatore are not flowering well. What should I do?"

Step 3 – Press Send

After typing your question, press the Enter key on your keyboard or click the Send button (usually shown as an arrow icon). The chatbot will take a few seconds to think and then show its answer on the screen above your question.

5.3 Understanding the Information the Chatbot Needs

Krishi Sahaya gives better answers when you include the following details in your question:

Information	Why it matters	Example
-------------	----------------	---------

Your district	The system uses district-specific weather and crop data	'I am in Thanjavur district'
Your crop	Different crops have different risk profiles and needs	'I grow rice' or 'I farm cotton'
Recent weather	Helps the ML model estimate yield risk more accurately	'It has been dry for 10 days'
Specific symptom	Helps the system find the most relevant ICAR document	'The leaves are turning yellow at the tips'
Time of season	Advice differs between sowing, growing, and harvest stages	'We are in the third month after sowing'

5.4 Reading and Interpreting the Response

Every response from Krishi Sahaya contains up to three parts:

Part 1 – Yield Risk Assessment

This section tells you how risky current conditions are for your crop's yield. The risk is shown as one of three labels:

Label	Meaning	What you should do
LOW RISK	Current weather and historical patterns suggest your crop should do well this season.	Continue normal farming practices. Stay alert for pest activity.
MEDIUM RISK	Some conditions are unfavourable. Your yield may be affected.	Follow the chatbot's specific advice on irrigation, fertiliser, or pest control. Monitor your crop closely.
HIGH RISK	Conditions are significantly unfavourable. There is a real chance of reduced yield.	Take immediate action based on the chatbot's recommendation. Consider contacting your nearest Krishi Vigyan Kendra or agricultural officer.

Part 2 – Recommendation

This is the main advice section. It is written in plain English and is based on your question, your weather conditions, and official ICAR guidelines. Read this carefully and follow the steps described. The advice may include:

- How often to irrigate and how much water to apply.
- Which fertiliser to use and when to apply it.
- Which pest or disease is likely causing a symptom and how to treat it.
- Information about a government scheme or MSP rate.

Part 3 – Sources

At the bottom of the response, you will see a list of source documents. These are the official ICAR guidebooks or government publications that the system used to form its advice. This tells you that the advice is not invented — it comes from tested, official knowledge. You can ask an agricultural officer to look up these documents if you want more detail.

5.5 A Sample Conversation

The following example shows how a conversation might look:

Text	
You type:	I farm rice in Thanjavur. It has been dry for 8 days and temperature is around 32 degrees. Should I irrigate now?
Krishi Sahaya responds:	Yield Risk: MEDIUM RISK (confidence: 78%) Recommendation:Based on current conditions in Thanjavur, rice under moderate moisture stress at 32°C requires attention. Irrigate now to bring soil moisture back to field capacity. Apply 5 cm of water if using flood irrigation. For drip-irrigated fields, run for 4 to 6 hours depending on soil type. Avoid irrigating during the hottest part of the day; early morning is preferred. Monitor for stem borer activity, which increases under water-stressed conditions. Sources: ICAR Irrigation Guide 2023, ICAR Pest Management – Tamil Nadu

5.6 Tips for Getting the Best Answers

1. Always mention your district by name. For example, write 'in Thanjavur' rather than just 'in my area'.
2. Mention your crop clearly. Write 'my rice crop' or 'my cotton fields' rather than just 'my crop'.
3. Describe symptoms precisely. Instead of 'my plants look sick', write 'the leaves are turning yellow at the tips with brown edges'.
4. One question at a time gives clearer answers. If you have multiple issues, ask about each one separately.
5. If the answer mentions a term you do not know, type 'What does [term] mean?' and the chatbot will explain it.
6. The chatbot's advice is based on patterns in historical data. For urgent or unusual situations, always confirm with a Krishi Vigyan Kendra or qualified agronomist.

5.7 Common Questions

Is Krishi Sahaya free to use?

Yes. The chatbot is completely free for all farmers.

Can I use it in Tamil?

The current version is designed for English questions. Tamil language support is planned for a future version.

How accurate is the yield risk prediction?

The model has an ROC-AUC score of 87%, which means it correctly ranks risk levels in roughly 87 out of 100 cases. It is a useful guide but should not replace direct observation of your fields or advice from an agricultural officer.

What if I do not know the exact temperature or humidity?

Give your best estimate. Even approximate values improve the prediction compared to leaving them out. You can find weather information for your district on weather apps or by calling your local Meteorological Department.

Will the system remember my previous questions?

Each session starts fresh. If you want to continue from a previous question, briefly summarise the context again in your new message.

What if the chatbot gives wrong advice?

No automated system is perfect. Always use the chatbot's advice as a starting point and verify with your local agricultural department or Krishi Vigyan Kendra before making large decisions like changing your irrigation schedule or applying a new pesticide.

Can I access it on my phone?

Yes. The chatbot works in any modern smartphone browser. Open your browser, type the address, and the page will adjust to fit your screen automatically.

What are the ICAR sources mentioned in the response?

ICAR stands for the Indian Council of Agricultural Research. These are official Indian government research organisations that publish guidelines on farming practices. When Krishi Sahaya cites an ICAR source, the advice comes directly from tested, peer-reviewed agricultural knowledge.
